BECOME A BACKEND DEVELOPER

HANINE KHALIL

khalil.hanine06@gmail.com

Hanine Khalil

Become A Backend Developer
(Part 2)

by
Hanine Khalil

This book is designed to guide beginners through Java fundamentals and core concepts,
and later dive into Spring Boot for building powerful backend applications.

Reach me at:

khalil.hanine06@gmail.com

Hanine Khalil

Contents

Chapter 1: Web & Backend Fundamentals **5**

1 Understanding the HTTP Protocol **5**
1.1 HTTP Methods . . . 6
1.2 HTTP Status Codes . . . 6

2 Understanding the HTTPS Protocol **6**

3 REST API Basics **7**
3.1 What is an API? . . . 7
3.2 Key Principles of REST . . . 8
3.3 How REST Uses HTTP . . . 8
3.4 Example REST API Request and Response . . . 8
3.5 Benefits of REST API . . . 9

4 Understanding JSON and XML **9**
4.1 What is JSON? . . . 9
4.2 What is XML? . . . 10
4.3 Comparison Between JSON and XML . . . 11

5 Authentication & Authorization **12**
5.1 Authentication . . . 12
5.2 Authorization . . . 12
5.3 Key Differences Between Authentication and Authorization . . . 13
5.4 Tokens in Authentication and Authorization . . . 13

6 MVC (Model-View-Controller) Architecture **14**
6.1 Components of MVC . . . 14
6.1.1 Model . . . 14
6.1.2 View . . . 14
6.1.3 Controller . . . 15
6.2 MVC Workflow . . . 15
6.3 Visual Diagram of MVC . . . 16
6.4 Real-World Analogy . . . 16
6.5 Benefits of MVC . . . 16

Chapter 2: Relational Databases **17**
2.1 What is a Relational Database? . . . 17
2.2 Why Use Relational Databases? . . . 17
2.3 Keys in Relational Databases . . . 17
2.4 Basic SQL Commands . . . 18
2.5 Relationships Between Tables . . . 20
2.6 Advantages of Relational Databases . . . 20
2.7 Real-World Example . . . 20

3 MySQL and PostgreSQL **21**
3.1 MySQL . . . 21
3.2 PostgreSQL . . . 22
3.3 MySQL vs PostgreSQL . . . 23
3.4 Real-World Analogy . . . 23

4 Joins, Indexes, and Normalization **23**
4.1 Joins . . . 23
4.2 Indexes . . . 24
4.3 Normalization . . . 24

5 ORM (Object-Relational Mapping) **25**
5.1 Why ORM is Important . . . 25
5.2 JPA (Java Persistence API) . . . 26
5.3 Hibernate . . . 26
5.4 Relationships in ORM . . . 27
5.5 Advantages of ORM . . . 27
5.6 When to Avoid ORM . . . 28

6 NoSQL Basics (MongoDB) **28**
6.1 Why NoSQL? . . . 28
6.2 MongoDB Overview . . . 28
6.3 Key MongoDB Commands . . . 29
6.4 Differences Between SQL and NoSQL . . . 29
6.5 When to Use MongoDB . . . 30
6.6 MongoDB Architecture Visualization . . . 30

Chapter 3: Spring Boot - Spring Core **31**
2.1 Key Concepts of Spring Core . . . 31
2.2 Inversion of Control (IoC) . . . 31
2.3 Dependency Injection (DI) . . . 32
2.4 Spring Container and Beans . . . 32
2.5 Bean Scopes . . . 33
2.6 Spring Configuration Methods . . . 33
2.7 Spring Core Workflow Visualization . . . 34
2.8 Benefits of Spring Core . . . 34

Chapter 4: Spring Boot - Fundamentals **35**
4.1 Spring Boot Architecture Overview . . . 35
4.1.1 Controller Layer . . . 35
4.1.2 Service Layer . . . 36
4.1.3 Repository Layer . . . 37
4.2 Entity Class . . . 38
4.3 Spring Boot REST Workflow . . . 38
4.4 Benefits of This Architecture . . . 40

5 Spring Data JPA **40**
5.1 What is JPA? . . . 40
5.2 How It Works . . . 40
5.3 Step-by-Step Example . . . 41
5.3.1 Define Entity Class . . . 41
5.3.2 Create Repository Interface . . . 41
5.3.3 Service Layer . . . 41
5.3.4 Controller Layer . . . 42
5.4 Custom Query Methods . . . 42
5.5 Spring Boot Application Setup . . . 43
5.6 Summary . . . 43

6 Spring Boot Validation **43**
6.1 Add Dependencies . . . 43
6.2 Annotate Your Entity . . . 44
6.3 Update the Controller . . . 44
6.4 Common Validation Annotations . . . 45
6.5 Optional: Global Error Handling . . . 45

7 Spring Boot Actuator **46**
7.1 Adding the Actuator Dependency . . . 46
7.2 Enable Actuator Endpoints in `application.properties` . . . 46
7.3 Accessing Actuator Endpoints . . . 46
7.4 Customizing Info Endpoint . . . 46
7.5 Health Endpoint Details . . . 47
7.6 Custom Health Indicator . . . 47
7.7 Securing Actuator Endpoints . . . 48

Chapter 5: Spring Security with JWT Authentication **49**
5.1 Overview of JWT Authentication . . . 49
5.2 Steps to Implement JWT Authentication in Spring Boot . . . 49
5.3 Add Dependencies . . . 49
5.4 Create User Entity and Repository . . . 50
5.5 Implement UserDetailsService . . . 50
5.6 JWT Utility Class . . . 51
5.7 Authentication Filter . . . 51
5.8 Authorization Filter . . . 52
5.9 Security Configuration . . . 53
5.10 Summary . . . 54

Chapter 6: Exception Handling and Logging in Spring Boot **55**
6.1 Why Exception Handling Matters . . . 55
6.2 Creating a Custom Exception . . . 55
6.3 Throwing the Exception . . . 55
6.4 Handling Exceptions Globally . . . 55
6.4.1 Returning JSON Responses . . . 56
6.5 Logging in Spring Boot . . . 56
6.5.1 Add Logger to Your Class . . . 57
6.5.2 Logging Levels . . . 57

6.6 Changing Log Level in application.properties 57
6.7 Example Error Response 57
6.8 Summary 58

Chapter 1: Web & Backend Fundamentals

1 Understanding the HTTP Protocol

HTTP (HyperText Transfer Protocol) is the foundation of communication on the web. It defines how data is requested and transferred between a **client** (e.g., browser) and a **server** (e.g., backend application).

When you type a URL in your browser or send an API request, you are using HTTP.

How HTTP Works

1. The **client** sends a request to the server (for example, requesting a web page).
2. The **server** processes the request and returns a response.
3. The response includes a **status code**, headers, and optionally a body (content).

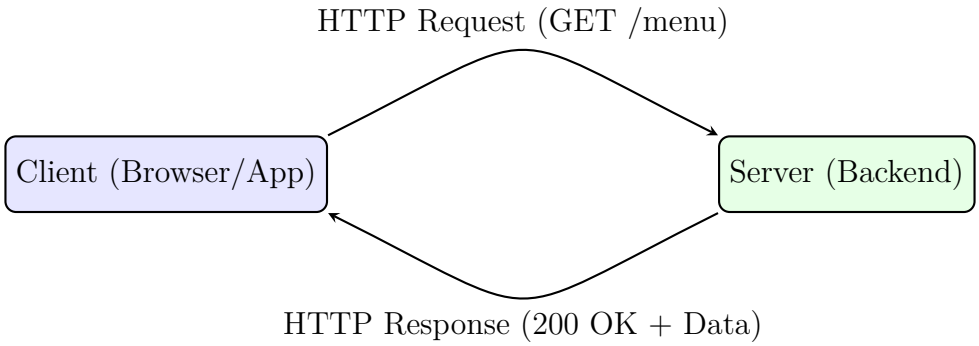
Real-Life Example: Imagine you are ordering food from a restaurant using a delivery app:

- The app (client) sends your order to the restaurant (server).
- The restaurant prepares the food and sends it back (response).
- You receive a notification: "Order confirmed" (**status code 200 OK**).

Similarly, in HTTP:

- **Client:** Browser or mobile app.
- **Request:** URL (e.g., `GET /menu`).
- **Server:** Web server hosting the menu.
- **Response:** Menu data and a status code (e.g., `200 OK`).

Visual Representation:



1.1 HTTP Methods

HTTP defines several request methods, each with a specific purpose:

- **GET** – Retrieve data from the server (e.g., fetch a list of users).
- **POST** – Send data to create a new resource (e.g., submit a form).
- **PUT** – Update an existing resource (replaces it entirely).
- **PATCH** – Partially update an existing resource (modify only specific fields).
- **DELETE** – Remove a resource from the server.
- **HEAD** – Same as GET, but retrieves only headers (no response body).
- **OPTIONS** – Shows supported HTTP methods for a resource.

1.2 HTTP Status Codes

HTTP responses include a **status code** to indicate the result of a request.

Category	**Description and Examples**
1xx (Informational)	Request received and still processing. Example: `100 Continue`
2xx (Success)	Request was successful. Examples: `200 OK`, `201 Created`
3xx (Redirection)	Client must take additional action. Examples: `301 Moved Permanently`, `302 Found`
4xx (Client Error)	Error caused by the client. Examples: `400 Bad Request`, `401 Unauthorized`, `404 Not Found`
5xx (Server Error)	Error caused by the server. Examples: `500 Internal Server Error`, `503 Service Unavailable`

2 Understanding the HTTPS Protocol

HTTPS (HyperText Transfer Protocol Secure) is the secure version of HTTP. It uses **SSL/TLS (Secure Sockets Layer / Transport Layer Security)** to encrypt communication between the client and server.

Why HTTPS is Important

- **Encryption:** Protects sensitive data (like passwords or payment details).
- **Authentication:** Ensures you are communicating with the real server.
- **Data Integrity:** Prevents data from being altered during transmission.
- **Trust:** Browsers mark HTTPS sites as secure (with a lock icon).

How HTTPS Works

1. Client requests a secure connection.
2. Server sends a digital certificate (SSL/TLS).
3. Client verifies the certificate using a trusted Certificate Authority.
4. Client and server establish an encrypted session.
5. HTTP communication then happens securely.

Example:

- Logging in to an online banking site.
- HTTPS encrypts your credentials so no one can intercept them.

3 REST API Basics

A **REST API (Representational State Transfer)** is an architectural style that allows communication between clients and servers using HTTP/HTTPS. It is widely used in web development because it is simple, scalable, and stateless.

3.1 What is an API?

- An **API (Application Programming Interface)** defines rules for software programs to interact.
- Example: A mobile app fetching user data from a backend server.

3.2 Key Principles of REST

1. **Client-Server Architecture:** The client (e.g. browser or app) and server are independent.
2. **Statelessness:** Each request is independent; the server does not remember previous requests.
3. **Resource-Based:** Everything is a resource (e.g., users, orders) identified by a unique URL.
4. **Uniform Interface:** REST APIs use standard HTTP methods (GET, POST, PUT, DELETE, etc.).
5. **Representation:** Data is usually returned in JSON format.

3.3 How REST Uses HTTP

REST APIs rely on HTTP methods already explained:

- `GET` – Retrieve resources.
- `POST` – Create new resources.
- `PUT/PATCH` – Update existing resources.
- `DELETE` – Remove resources.

3.4 Example REST API Request and Response

```
GET /users/1 HTTP/1.1
Host: api.example.com
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json

{
    "id": 1,
    "name": "Hanine",
    "email": "hanine@example.com"
}
```

3.5 Benefits of REST API

Response:

- **Scalable:** Easy to add new endpoints.
- **Language Independent:** Can be used by any programming language.
- **Simple and Flexible:** Uses standard HTTP methods and JSON.
- **Widely Adopted:** The most common style for APIs.

4 Understanding JSON and XML

When building web applications or APIs, data is exchanged between clients (e.g., browsers or mobile apps) and servers. Two of the most widely used formats for this data exchange are **JSON (JavaScript Object Notation)** and **XML (eXtensible Markup Language)**.

4.1 What is JSON?

JSON (JavaScript Object Notation) is a lightweight data-interchange format that is easy for humans to read and write, and easy for machines to parse and generate. It is widely used in modern web development because it is simple and efficient.

Key Features of JSON:

- **Lightweight:** Minimal syntax with less overhead compared to XML.
- **Human-readable:** Uses a simple key-value pair format.
- **Language Independent:** Though derived from JavaScript, it is supported by almost all programming languages.
- **Commonly used in APIs:** Most REST APIs return data in JSON format.

JSON Syntax:

- Data is represented as **key-value pairs**.
- Keys are always strings enclosed in double quotes.
- Values can be strings, numbers, objects, arrays, booleans, or null.

Example:

```
{
    "id": 1,
    "name": "Hanine",
    "email": "hanine@example.com",
    "skills": ["Java", "Spring Boot", "SQL"],
    "isActive": true
}
```

In this example:

- `id` → number
- `name` → string
- `skills` → array of strings
- `isActive` → boolean

4.2 What is XML?

XML (eXtensible Markup Language) is another format used to store and transport data. It is more verbose than JSON and uses custom tags to describe data.

Key Features of XML:

- **Self-descriptive:** Uses custom tags to describe data meaning.
- **Extensible:** You can define your own tags.
- **Supports metadata:** Attributes can provide extra details.
- **Widely used in enterprise systems:** Common in legacy applications, SOAP APIs, and configurations.

XML Syntax:

- Data is enclosed within opening and closing tags.
- Tags can have attributes.
- XML is case-sensitive.

Example:

```
<User id="1">
    <Name>Hanine</Name>
    <Email>hanine@example.com</Email>
    <Skills>
        <Skill>Java</Skill>
        <Skill>Spring Boot</Skill>
        <Skill>SQL</Skill>
    </Skills>
    <IsActive>true</IsActive>
</User>
```

In this example:

- The root element is `<User>`.
- `id="1"` is an attribute.
- `<Skills>` contains multiple nested `<Skill>` elements.

4.3 Comparison Between JSON and XML

Aspect	**JSON**	**XML**
Syntax	Key-value pairs with braces { } and brackets [ ]	Tag-based structure with opening and closing tags
Readability	Simple and compact	More verbose and harder to read
Data Types	Supports strings, numbers, booleans, arrays, objects	All values are text; additional parsing needed for types
Attributes	Not supported (all are keys)	Supports attributes within tags
Usage	REST APIs, modern web apps, configuration files	Enterprise applications, SOAP APIs, legacy systems
Parsing Speed	Faster	Slower compared to JSON

5 Authentication & Authorization

In web development, securing applications is crucial. Two fundamental concepts are:

- **Authentication:** Proving *who you are*.
- **Authorization:** Determining *what you are allowed to do*.

Although they are related, they are not the same.

5.1 Authentication

Authentication is the process of verifying the identity of a user or system. It ensures that the person or system trying to access the application is who they claim to be.

How it Works:

1. The user provides credentials (e.g., username and password).
2. The server checks these credentials against stored records.
3. If the credentials are valid, the server confirms the user's identity.

Common Authentication Methods:

- **Username & Password** – The most common method.
- **Multi-Factor Authentication (MFA)** – Adds extra security (e.g., OTP via SMS or email).
- **OAuth / Social Login** – Login with Google, Facebook, GitHub, etc.
- **Biometric Authentication** – Using fingerprint or facial recognition.

Real-Life Example: When you enter a building, authentication is like showing your ID card at the entrance to prove your identity.

5.2 Authorization

Authorization happens **after authentication**. Once the user's identity is verified, the system determines what actions or resources the user is allowed to access.

How it Works:

1. After logging in (authentication), the server assigns roles or permissions to the user.
2. The system checks these permissions every time the user tries to perform an action.

Common Authorization Methods:

- **Role-Based Access Control (RBAC):** Users are assigned roles (e.g., Admin, Editor, Viewer).
- **Permission-Based Access Control:** Permissions are assigned to users or roles for fine-grained control.
- **Token-Based Authorization:** After login, the server gives a token (like a key) that is used to access resources.

Real-Life Example: After showing your ID card at the building entrance (authentication), you are only allowed to enter specific rooms (authorization) based on your access level.

5.3 Key Differences Between Authentication and Authorization

Aspect	**Authentication**	**Authorization**
Definition	Verifying the identity of a user	Determining what resources or actions the user can access
Purpose	To confirm "Who are you?"	To confirm "What can you do?"
Process	Always comes first	Happens after authentication
Example	Logging in with username and password	Allowing only admins to delete user accounts
Data	Credentials (username, password, biometrics)	Roles, permissions, access rules

5.4 Tokens in Authentication and Authorization

In modern web development, authentication and authorization often use **tokens**:

- After a successful login, the server generates a token (e.g., JWT – JSON Web Token).
- The client stores this token and includes it in every request.
- The server verifies the token to identify the user and their permissions.

6 MVC (Model-View-Controller) Architecture

MVC is a popular software design pattern used to organize code in web and backend applications. It separates an application into three main components:

- **Model** – Manages data and business logic.
- **View** – Handles the user interface and presentation.
- **Controller** – Acts as a bridge between the Model and the View.

This separation improves maintainability, scalability, and makes it easier for developers to work on different parts of an application independently.

6.1 Components of MVC

6.1.1 Model

- Represents the **data** and the rules for manipulating that data.
- Handles database operations, business logic, and state management.
- Does not deal with how the data is displayed.
- Example: A `User` model that retrieves user details from a database.

Example (Java-like pseudo-code):

```
class UserModel {
    private String name;
    private String email;

    public UserModel(String name, String email) {
        this.name = name;
        this.email = email;
    }

    public String getName() { return name; }
    public String getEmail() { return email; }
}
```

6.1.2 View

- Displays the data to the user (UI).
- Gets the data from the Controller and formats it for presentation.
- Should not contain business logic.
- Example: An HTML page or a frontend template displaying user data.

```
<!-- View (HTML template) -->
<h1>User Profile</h1>
<p>Name: Hanine</p>
<p>Email: hanine@example.com</p>
```

6.1.3 Controller

Example:

- Manages communication between the View and the Model.
- Accepts input (e.g., from the browser), processes it, and updates the Model or View accordingly.
- Example: A controller method that retrieves user data from the Model and passes it to the View.

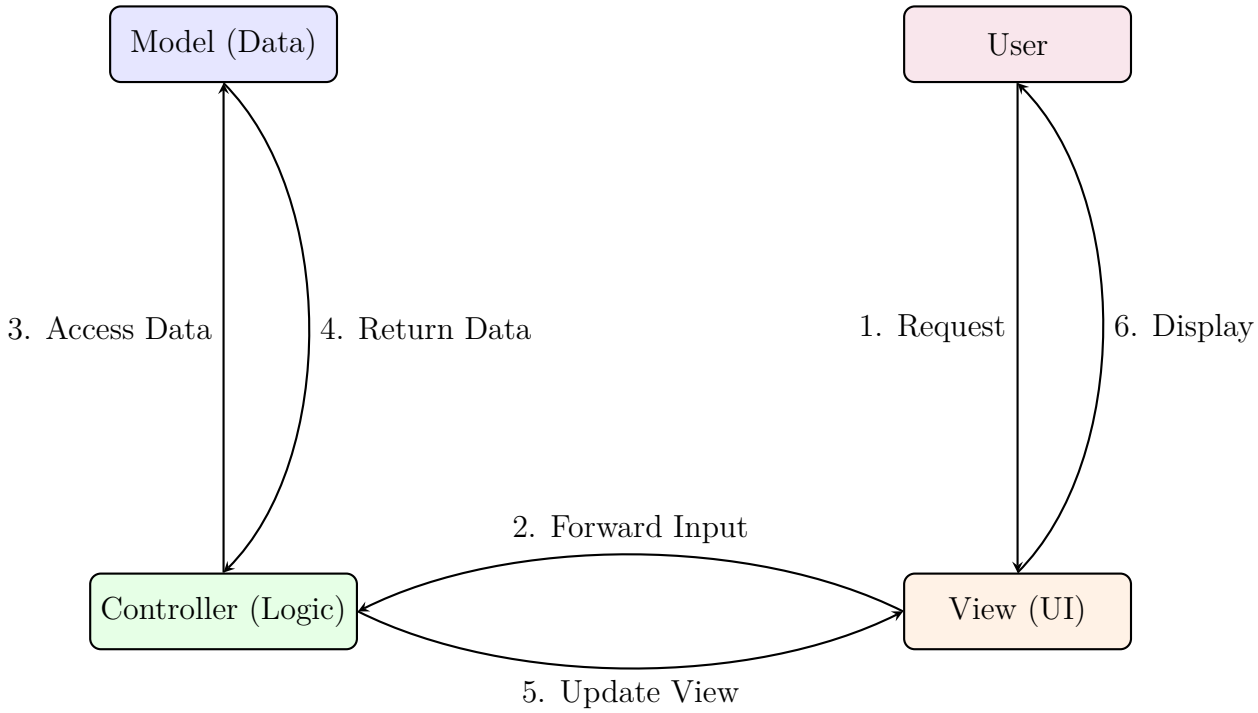
Example (Java-like pseudo-code):

```java
class UserController {
    public void showUserProfile() {
        UserModel user = new UserModel("Hanine", "hanine@example.
            com");
        System.out.println("Name: " + user.getName());
        System.out.println("Email: " + user.getEmail());
    }
}
```

6.2 MVC Workflow

1. The **User** interacts with the **View** (e.g., clicks a button or submits a form).
2. The **Controller** receives the request and processes it.
3. The **Controller** communicates with the **Model** to fetch or update data.
4. The **Model** returns the data to the **Controller**.
5. The **Controller** sends the data to the **View** for display.
6. The **View** presents the updated information to the **User**.

6.3 Visual Diagram of MVC



6.4 Real-World Analogy

Think of MVC like a restaurant:

- **Model (Kitchen):** Prepares the food (data).
- **Controller (Waiter):** Takes your order, delivers it to the kitchen, and brings the food back.
- **View (Menu/Table):** Displays the options and serves the prepared dish to you.
- **User (Customer):** Orders food and consumes it.

6.5 Benefits of MVC

- **Separation of Concerns:** Each component has a single responsibility.
- **Maintainability:** Easier to modify one layer without breaking the others.
- **Reusability:** Models and Views can be reused across different Controllers.
- **Scalability:** Well-suited for large applications with multiple developers.

Chapter 2: Relational Database

Relational Databases are one of the most widely used types of databases. They organize data into **tables** (also called relations) that are connected using **keys**. SQL (**Structured Query Language**) is the standard language used to interact with relational databases.

2.1 What is a Relational Database?

A **relational database** stores data in a structured format using tables, rows, and columns:

- **Table:** Represents an entity (e.g., Users, Orders).
- **Row (Record):** Represents a single entry in the table.
- **Column (Field):** Represents an attribute of the entity.

Example: Users Table

ID	Name	Email
1	Hanine	hanine@example.com
2	Ali	ali@example.com
3	Sara	sara@example.com

Each row represents a single user, and each column describes a specific property of that user.

2.2 Why Use Relational Databases?

- **Structured Data:** Well-organized data in tables.
- **Relationships:** You can link tables using keys (e.g., Users and Orders).
- **ACID Properties:** Ensure reliability (Atomicity, Consistency, Isolation, Durability).
- **Scalability:** Efficient for handling large amounts of structured data.
- **Standard Language:** SQL is a universal language for querying databases.

2.3 Keys in Relational Databases

- **Primary Key (PK):** A unique identifier for each record in a table.
- **Foreign Key (FK):** A reference to a primary key in another table (used to create relationships).

Example:

Users Table		
UserID (PK)	**Name**	**Email**
1	Hanine	hanine@example.com
2	Ali	ali@example.com

Orders Table		
OrderID (PK)	**UserID (FK)**	**Amount**
101	1	$50
102	2	$30

Here, the `UserID` in the `Orders` table is a **foreign key** linking each order to a user.

2.4 Basic SQL Commands

SQL is used to manage data in a relational database. Here are the most common commands with their syntax and examples:

- **`CREATE TABLE`** – Create a new table.

  Syntax:

```
CREATE TABLE table_name (
    column1 datatype constraint,
    column2 datatype constraint,
    ...
);
```

  Example:

```
CREATE TABLE Users (
    UserID INT PRIMARY KEY,
    Name VARCHAR(50),
    Email VARCHAR(100)
);
```

- **`INSERT INTO`** – Add new records to a table.

  Syntax:

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

  Example:

```
INSERT INTO Users (UserID, Name, Email)
VALUES (1, 'Hanine', 'hanine@example.com');
```

- **SELECT** – Retrieve data from a table.

  Syntax:

```
SELECT column1, column2, ...
FROM table_name;
```

  Example:

```
SELECT * FROM Users;
```

- **UPDATE** – Modify existing data.

  Syntax:

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

  Example:

```
UPDATE Users
SET Email = 'hanine.new@example.com'
WHERE UserID = 1;
```

- **DELETE** – Remove data from a table.

  Syntax:

```
DELETE FROM table_name
WHERE condition;
```

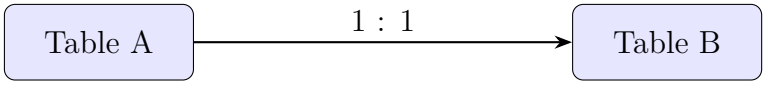
  Example:

```
DELETE FROM Users
WHERE UserID = 1;
```

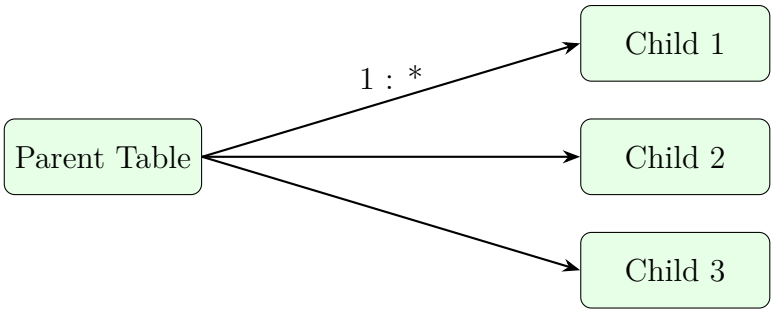
2.5 Relationships Between Tables

Relational databases support several types of relationships:

- **One-to-One:** Each record in one table relates to exactly one record in another.



- **One-to-Many:** A record in one table can relate to multiple records in another (most common).



- **Many-to-Many:** Requires a **junction table** to connect records between two tables.

2.6 Advantages of Relational Databases

- Easy to query with SQL.
- Strong data integrity and consistency.
- Support for complex relationships between data.
- Widely used and supported by many database management systems (e.g., MySQL, PostgreSQL, SQL Server).

2.7 Real-World Example

In an e-commerce platform:

- The `Users` table stores customer information.
- The `Orders` table stores order details.
- Foreign keys link each order to the user who placed it.

This allows the platform to fetch a user's orders quickly using SQL queries.

3 MySQL and PostgreSQL

Relational Database Management Systems (RDBMS) are the backbone of most backend applications. Among the most popular RDBMS are **MySQL** and **PostgreSQL**. Both are open-source, reliable, and widely used for managing relational data, but they have different strengths.

3.1 MySQL

MySQL is one of the most widely used relational databases, especially in web development.

Key Features:

- **Open-Source and Free:** Owned by Oracle but has a free community edition.
- **Performance-Focused:** Optimized for read-heavy workloads.
- **Widely Supported:** Works with many tools like PHP, Node.js, and Java.
- **Replication Support:** Built-in master-slave replication for scalability.
- **Storage Engines:** Supports multiple engines (e.g., InnoDB for transactions, MyISAM for speed).

When to Use MySQL:

- Web applications (e.g., WordPress, e-commerce platforms).
- Applications with high read operations.
- Systems where simplicity and speed are priorities.

Example Command:

```sql
-- Create a table for storing users
CREATE TABLE Users (
    UserID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(50),
    Email VARCHAR(100) UNIQUE
);
```

3.2 PostgreSQL

PostgreSQL is an advanced, open-source database system known for its robustness and flexibility.

Key Features:

- **Open-Source and Free:** Managed by a global community.
- **ACID Compliant:** Ensures reliable transactions.
- **Advanced Data Types:** Supports JSON, XML, arrays, and even custom data types.
- **Full-Text Search:** Powerful built-in search functionality.
- **Extensibility:** Users can add custom functions and operators.
- **Concurrency:** Uses Multi-Version Concurrency Control (MVCC) for high performance.

When to Use PostgreSQL:

- Applications that require complex queries.
- Data analytics and reporting tools.
- APIs and systems needing JSON and NoSQL-like capabilities.

Example Command:

```sql
-- Create a table with JSON support
CREATE TABLE Products (
    ProductID SERIAL PRIMARY KEY,
    Name VARCHAR(100),
    Details JSONB
);
```

3.3 MySQL vs PostgreSQL

Feature	MySQL	PostgreSQL
Performance	Faster for read-heavy workloads	Optimized for complex queries and analytics
Data Types	Limited (mainly standard SQL types)	Extensive (JSON, arrays, custom types)
Transactions	Good, but less advanced	Fully ACID-compliant with MVCC
Community	Very large and widely used in web dev	Strong in enterprise and advanced dev
Extensibility	Limited	Highly extensible (custom functions)
Use Case	Web apps, e-commerce, CMS	APIs, analytics, financial apps

3.4 Real-World Analogy

- **MySQL:** Like a fast-food restaurant—simple, fast, and efficient for common use cases.
- **PostgreSQL:** Like a fine dining restaurant—more advanced options, better customization, but may require more setup.

4 Joins, Indexes, and Normalization

Relational databases provide powerful tools to efficiently store, retrieve, and organize data. Three of the most important concepts for working with SQL databases are **Joins**, **Indexes**, and **Normalization**.

4.1 Joins

A **JOIN** in SQL is used to combine rows from two or more tables based on a related column.

Types of Joins:

- **INNER JOIN:** Returns only rows that have matching values in both tables.
- **LEFT JOIN (or LEFT OUTER JOIN):** Returns all rows from the left table and the matching rows from the right table. Non-matching rows from the right table will be `NULL`.
- **RIGHT JOIN (or RIGHT OUTER JOIN):** Returns all rows from the right table and the matching rows from the left table.

- **FULL JOIN (or FULL OUTER JOIN):** Returns rows when there is a match in one of the tables; unmatched rows are filled with `NULL`.
- **CROSS JOIN:** Returns a Cartesian product of both tables (every row from table A combined with every row from table B).

Example (INNER JOIN):

```sql
SELECT Users.Name, Orders.OrderDate
FROM Users
INNER JOIN Orders ON Users.UserID = Orders.UserID;
```

This query fetches only users who have placed an order.

4.2 Indexes

An **Index** is a database structure that improves the speed of data retrieval operations on a table. It works similarly to an index in a book: instead of reading every page (full table scan), the database uses the index to find the data quickly.

Key Points:

- Speeds up `SELECT` queries.
- Slows down `INSERT`, `UPDATE`, and `DELETE` because the index must also be updated.
- Uses extra storage space.

Creating an Index:

```sql
-- Create an index on the Email column
CREATE INDEX idx_users_email ON Users (Email);
```

When to Use Indexes:

- On columns used frequently in `WHERE` conditions.
- On columns used in `JOIN` operations.
- On columns that require sorting or filtering often.

4.3 Normalization

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity.

Goals of Normalization:

- Avoid duplicate data.
- Ensure logical data storage.
- Simplify database maintenance.

Common Normal Forms:

- **1NF (First Normal Form):**
  - Each column must contain atomic (indivisible) values.
  - Each row must be unique.
- **2NF (Second Normal Form):**
  - Must be in 1NF.
  - All non-key columns must depend on the entire primary key.
- **3NF (Third Normal Form):**
  - Must be in 2NF.
  - No column should depend on another non-key column (removes transitive dependency).

Example of Normalization:

- **Unnormalized Table:**

UserID	Name	Orders
1	Hanine	Pizza, Burger
2	Ali	Pasta

- **Normalized (1NF):**

UserID	Name	Order
1	Hanine	Pizza
1	Hanine	Burger
2	Ali	Pasta

5 ORM (Object-Relational Mapping)

ORM is a programming technique that allows developers to interact with a database using objects instead of writing raw SQL queries. In Java, the most popular ORM tools are **Hibernate** and **JPA (Java Persistence API)**.

5.1 Why ORM is Important

- **Abstraction:** Developers work with Java objects instead of SQL queries.
- **Productivity:** Reduces boilerplate code for database operations (e.g., `INSERT`, `UPDATE`).
- **Portability:** Code is database-agnostic and works across multiple database systems.
- **Maintainability:** Easier to modify database structure without changing a lot of application logic.
- **Relationship Mapping:** Handles table relationships (one-to-one, one-to-many, many-to-many) automatically.

5.2 JPA (Java Persistence API)

- JPA is a **standard specification** in Java for ORM.
- It defines a set of annotations and interfaces for mapping Java objects to database tables.
- JPA does not implement ORM itself; it needs a provider such as Hibernate or EclipseLink.

Example Entity (JPA):

```
import jakarta.persistence.*;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(unique = true)
    private String email;

    // Getters and setters
}
```

This maps the `User` class to a database table named `users`.

5.3 Hibernate

- Hibernate is the most widely used **JPA implementation**.
- It provides additional features like caching, lazy loading, and query language (**HQL – Hibernate Query Language**).
- Hibernate handles the database connection, SQL generation, and transaction management.

Saving an Object with Hibernate:

```
User user = new User();
user.setName("Hanine");
user.setEmail("hanine@example.com");

EntityManagerFactory emf = Persistence
            .createEntityManagerFactory("my-pu");

EntityManager em = emf.createEntityManager();
```

```
em.getTransaction().begin();
em.persist(user); // Hibernate generates the INSERT query
em.getTransaction().commit();
em.close();
```

Hibernate automatically translates this into:

```
INSERT INTO users (name, email) VALUES ('Hanine', 'hanine@ex.com');
```

5.4 Relationships in ORM

ORM frameworks can map table relationships to Java objects:

- **@OneToOne** – One object is linked to exactly one other object.
- **@OneToMany** – One object is linked to multiple objects.
- **@ManyToOne** – Many objects link to one object (inverse of OneToMany).
- **@ManyToMany** – Many objects are linked to many other objects through a join table.

Example (One-to-Many):

```
@Entity
public class User {

    @Id
    @GeneratedValue
    private Long id;

    @OneToMany(mappedBy = "user")
    private List<Order> orders;
}
```

5.5 Advantages of ORM

- No need to write repetitive SQL queries.
- Automatically maps Java classes to database tables.
- Built-in transaction and caching support.
- Supports database migrations with minimal code changes.

5.6 When to Avoid ORM

Although ORM is powerful, it might not be suitable in cases where:

- You need very complex queries or database-specific optimizations.
- Performance is critical and raw SQL is more efficient.
- You are working with a legacy database with poor normalization.

6 NoSQL Basics (MongoDB)

NoSQL (Not Only SQL) databases are designed to handle large volumes of unstructured or semi-structured data. Unlike relational databases, NoSQL databases do not rely on fixed schemas or tables, making them highly flexible and scalable.

One of the most popular NoSQL databases is **MongoDB**.

6.1 Why NoSQL?

- **Scalability:** Easily handles massive amounts of data by scaling horizontally across multiple servers.
- **Flexibility:** Does not require predefined schemas; data can have dynamic fields.
- **High Performance:** Optimized for read and write operations on large datasets.
- **Big Data and Real-Time Applications:** Ideal for applications like social media, e-commerce, analytics, and IoT.

6.2 MongoDB Overview

- **Document-Oriented:** Stores data as **documents** in JSON-like format called BSON (Binary JSON).
- **Collections:** Equivalent to tables in SQL databases.
- **Documents:** Equivalent to rows/records, but they can have different fields.
- **Dynamic Schema:** Fields can vary between documents in the same collection.

Example Document in MongoDB:

```
{
    "_id": 1,
    "name": "Hanine",
    "email": "hanine@example.com",
    "skills": ["Java", "Spring Boot", "MongoDB"],
    "isActive": true
}
```

6.3 Key MongoDB Commands

Create a Database and Collection:

```
use myDatabase          // Create or switch to database
db.createCollection("users")
```

Insert a Document:

```
db.users.insertOne({
    name: "Hanine",
    email: "hanine@example.com"
})
```

Find Documents:

```
db.users.find()                      // Retrieve all documents
db.users.find({ name: "Hanine" })    // Filter by name
```

Update a Document:

```
db.users.updateOne(
    { name: "Hanine" },
    { $set: { email: "hanine.new@example.com" } }
)
```

Delete a Document:

```
db.users.deleteOne({ name: "Hanine" })
```

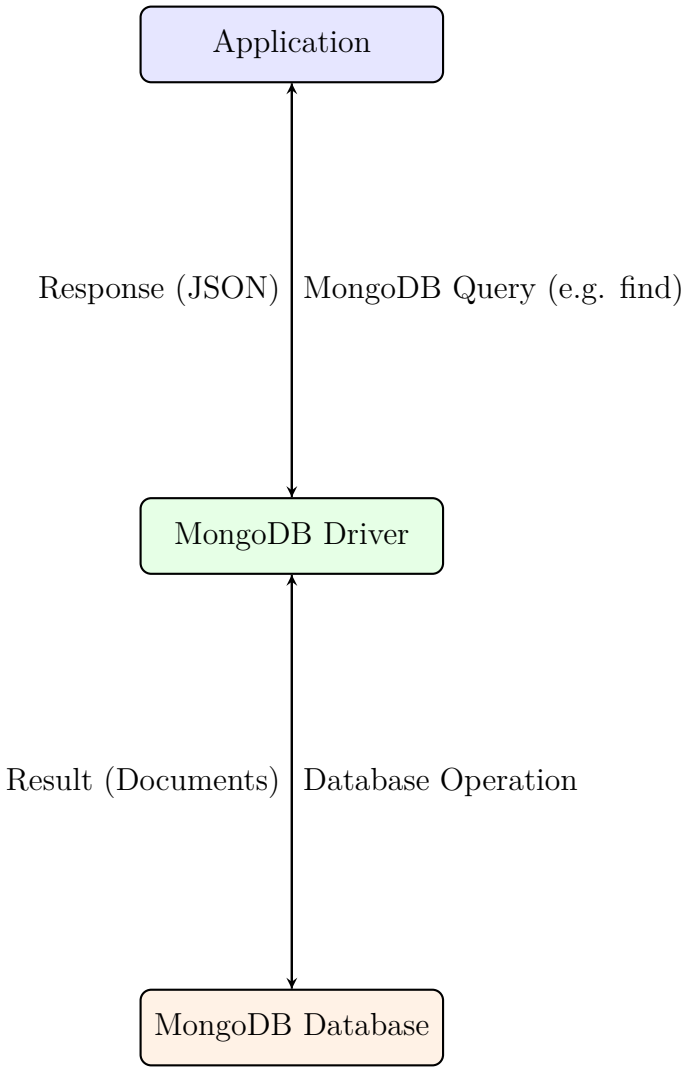
6.4 Differences Between SQL and NoSQL

Aspect	SQL (Relational DB)	NoSQL (MongoDB)
Data Model	Tables with rows and columns	Collections with JSON-like documents
Schema	Fixed schema required	Schema-less (dynamic structure)
Scalability	Vertical (add more CPU/RAM)	Horizontal (add more servers)
Query Language	SQL	MongoDB Query Language (MQL)
Transactions	Strong ACID support	Eventual consistency (but supports ACID in recent versions)
Use Case	Banking, ERP, structured data	Big data, real-time analytics, flexible data

6.5 When to Use MongoDB

- When data structure changes frequently (dynamic schema).
- For real-time analytics or large-scale data.
- When you need fast development and flexibility.
- In applications that handle semi-structured data such as JSON.

6.6 MongoDB Architecture Visualization



Chapter 3: Spring Boot - Spring Core

Spring Core is the foundation of the entire Spring Framework. It provides the core features such as dependency injection, inversion of control (IoC), and the container that manages the lifecycle of objects in your application.

Understanding Spring Core is critical because it explains how Spring creates and manages objects, wires dependencies together, and provides a flexible configuration model.

2.1 Key Concepts of Spring Core

- **IoC (Inversion of Control):** The control of creating and managing objects is transferred from the developer to the Spring container.
- **Dependency Injection (DI):** Instead of creating objects manually, dependencies are injected into classes by Spring.
- **Spring Container:** A container that creates and manages all beans (objects) defined in the application.
- **Beans:** Objects that are managed by the Spring container.
- **Configuration:** The process of telling Spring how to create and manage beans (via XML, Java annotations, or Java configuration classes).

2.2 Inversion of Control (IoC)

Inversion of Control means that instead of your application creating and controlling objects, the control is inverted: the **Spring container** is responsible for this.

Without IoC (Traditional Approach):

```
class UserService {
    private UserRepository userRepository;

    public UserService() {
        this.userRepository = new UserRepository(); // Manually
           creating dependency
    }
}
```

With IoC (Spring):

```
@Component
class UserRepository {}

@Component
class UserService {
    @Autowired
    private UserRepository userRepository; // Dependency is
       injected by Spring
}
```

2.3 Dependency Injection (DI)

Dependency Injection is the mechanism by which Spring supplies the required dependencies (beans) to a class.

Types of Dependency Injection:

- **Constructor Injection** – Dependencies are provided through the class constructor.
- **Setter Injection** – Dependencies are provided through setter methods.
- **Field Injection** – Dependencies are injected directly into class fields using annotations like `@Autowired`.

Example: Constructor Injection

```
@Component
class UserRepository {}

@Component
class UserService {
    private final UserRepository userRepository;

    @Autowired
    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
}
```

2.4 Spring Container and Beans

The **Spring Container** is responsible for:

- Creating beans (objects).
- Injecting dependencies.
- Managing the lifecycle of beans.

Types of Spring Containers:

- **BeanFactory:** The simplest container, provides basic DI support.
- **ApplicationContext:** An advanced container that supports internationalization, event propagation, and more (most commonly used).

2.5 Bean Scopes

Bean scope defines how Spring creates and shares bean instances.

- **Singleton (default):** Only one instance is created and shared.
- **Prototype:** A new instance is created every time it is requested.
- **Request (Web):** A new bean instance per HTTP request.
- **Session (Web):** A new bean instance per HTTP session.

Example: Defining Bean Scope

```
@Component
@Scope("prototype")
class UserService {
    // This bean will have a new instance every time it is
       requested
}
```

2.6 Spring Configuration Methods

Spring allows multiple ways to configure beans and dependencies:

- **XML-Based Configuration:** Using `applicationContext.xml`.
- **Annotation-Based Configuration:** Using annotations like `@Component`, `@Autowired`, and `@Configuration`.
- **Java-Based Configuration:** Using `@Bean` methods inside a configuration class.

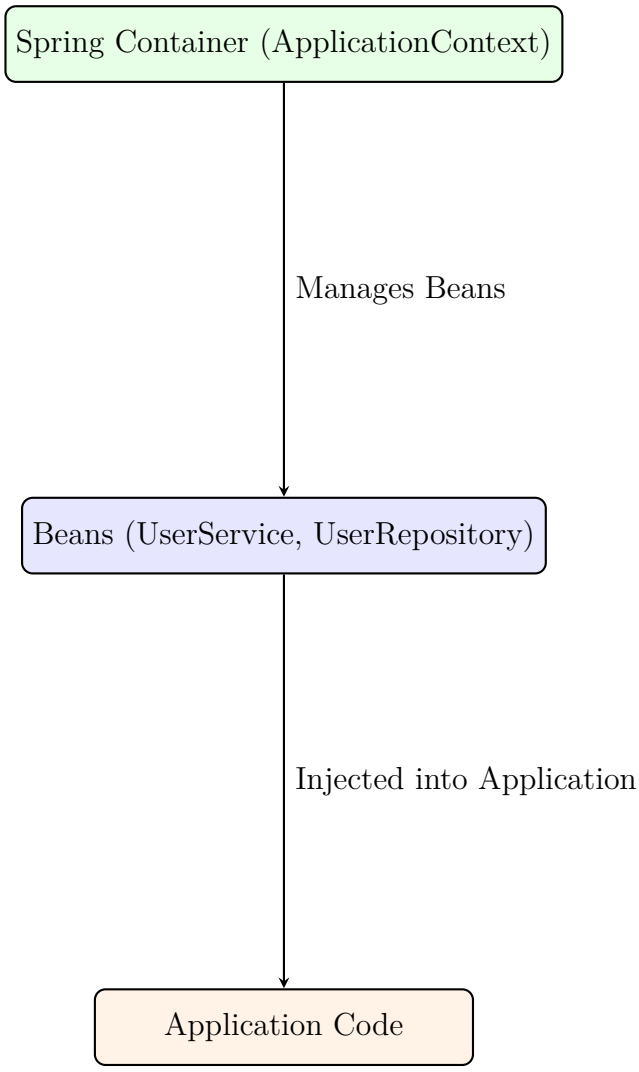
Example: Java-Based Configuration

```
@Configuration
public class AppConfig {

    @Bean
    public UserRepository userRepository() {
        return new UserRepository();
    }

    @Bean
    public UserService userService() {
        return new UserService(userRepository());
    }
}
```

2.7 Spring Core Workflow Visualization



2.8 Benefits of Spring Core

- Reduces boilerplate code for object creation and wiring.
- Promotes loose coupling between classes.
- Provides easy testing and maintainability.
- Offers flexibility through multiple configuration options.

Chapter 4: Spring Boot - Fundamentals

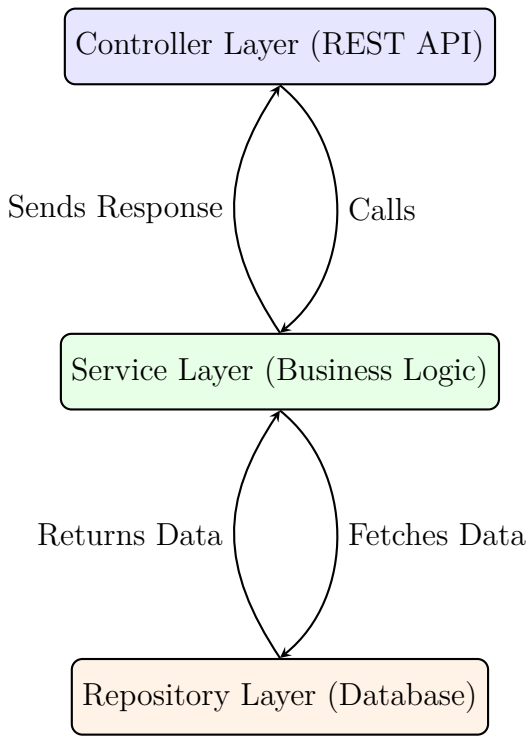
Spring Boot simplifies the development of Spring applications by providing pre-configured defaults, auto-configuration, and embedded servers (like Tomcat). With Spring Boot, you can create production-ready applications quickly without worrying about complex XML configurations.

4.1 Spring Boot Architecture Overview

Spring Boot applications follow a layered architecture that separates concerns:

1. **Controller Layer:** Handles HTTP requests and responses.
2. **Service Layer:** Contains business logic.
3. **Repository Layer:** Interacts with the database.

This separation makes applications easier to maintain, test, and scale.



4.1.1 Controller Layer

The **controller layer** is responsible for handling incoming HTTP requests, processing them (with the help of the service layer), and sending back responses. Controllers in Spring Boot are annotated with `@RestController` or `@Controller`.

Key Annotations:

- `@RestController` – Combines `@Controller` and `@ResponseBody` to directly return JSON or XML responses.
- `@RequestMapping` – Maps HTTP requests to handler methods.
- `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping` – Shortcut annotations for specific HTTP methods.
- `@PathVariable` – Extracts dynamic values from the URL.
- `@RequestBody` – Reads data from the request body.

Example: User Controller

```
@RestController
@RequestMapping("/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return userService.getUserById(id);
    }

    @PostMapping
    public User createUser(@RequestBody User user) {
        return userService.saveUser(user);
    }
}
```

Here:

- `GET /users/1` → Fetches a user by ID.
- `POST /users` → Creates a new user.

4.1.2 Service Layer

The **service layer** contains the application's business logic. It is annotated with **`@Service`**, making it a Spring-managed bean.

Responsibilities of the Service Layer:

- Validate data.
- Implement business rules.
- Coordinate between controller and repository.
- Transform or process data before sending it back.

Example: User Service

```
@Service
public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User getUserById(Long id) {
        return userRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("User not
                    found"));
    }

    public User saveUser(User user) {
        return userRepository.save(user);
    }
}
```

Here:

- The service calls `UserRepository` for data.
- Handles errors (e.g. user not found).

4.1.3 Repository Layer

The **repository layer** interacts with the database. In Spring Boot, this is typically implemented using Spring Data JPA, which provides built-in CRUD operations.

Key Annotation:

- `@Repository` – Marks the class as a repository bean.
- `JpaRepository<T, ID>` – Provides ready-to-use database operations.

Example: User Repository

```
@Repository
public interface UserRepository extends JpaRepository<User, Long>
    {
    // Spring Data JPA will auto-implement common methods
    // You can also define custom queries if needed
}
```

Here, Spring automatically generates methods like:

- `findById(Long id)`
- `save(User user)`
- `deleteById(Long id)`
- `findAll()`

4.2 Entity Class

The repository layer works with entity classes mapped to database tables.

Example: User Entity

```
@Entity
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;

    // Getters and Setters
}
```

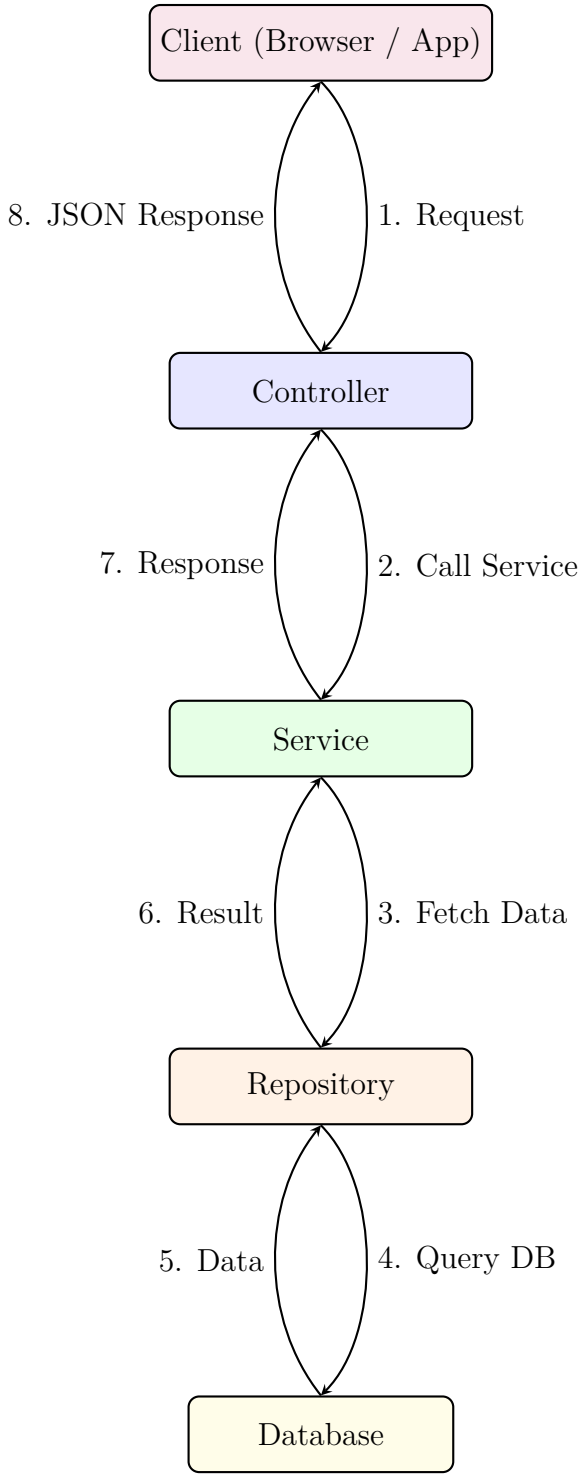
This class:

- Maps to a `User` table in the database.
- Has an auto-generated primary key (`id`).

4.3 Spring Boot REST Workflow

1. The client (e.g. browser or mobile app) sends a request to the **controller**.
2. The controller forwards the request to the **service layer**.
3. The service layer fetches or manipulates data via the **repository layer**.
4. The repository interacts with the database and returns the data.

5. The service layer applies business logic if needed.
6. The controller sends the final response back to the client.



4.4 Benefits of This Architecture

- **Separation of Concerns:** Each layer has a clear responsibility.
- **Easier Maintenance:** Changes in one layer do not affect others.
- **Reusability:** Services and repositories can be reused across different controllers.
- **Testability:** Each layer can be tested independently.

5 Spring Data JPA

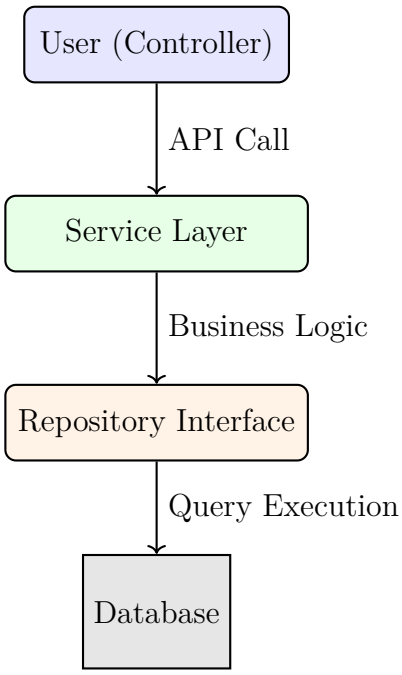
Spring Data JPA simplifies the data access layer in Spring applications by reducing boilerplate code required to interact with a relational database. It integrates seamlessly with Spring Boot and is built on top of JPA (Java Persistence API).

5.1 What is JPA?

JPA (Java Persistence API) is a specification for accessing, persisting, and managing data between Java objects and relational databases.

Spring Data JPA is an implementation of this specification that provides ready-to-use repository interfaces and methods.

5.2 How It Works



5.3 Step-by-Step Example

Let's implement a simple `User` entity using Spring Data JPA.

5.3.1 Define Entity Class

Listing 1: User Entity

```
import jakarta.persistence.*;

@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;

    // Getters and Setters
}
```

5.3.2 Create Repository Interface

Spring Data JPA will automatically provide basic CRUD operations without implementing methods.

Listing 2: User Repository

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long>
    {
    // Custom methods can be defined here
}
```

5.3.3 Service Layer

Listing 3: User Service

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public List<User> getAllUsers() {
        return userRepository.findAll();
```

```
    }

    public User createUser(User user) {
        return userRepository.save(user);
    }

    public void deleteUser(Long id) {
        userRepository.deleteById(id);
    }
}
```

5.3.4 Controller Layer

Listing 4: User Controller

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/users")
public class UserController {

    @Autowired
    private UserService userService;

    @GetMapping
    public List<User> getUsers() {
        return userService.getAllUsers();
    }

    @PostMapping
    public User createUser(@RequestBody User user) {
        return userService.createUser(user);
    }

    @DeleteMapping("/{id}")
    public void deleteUser(@PathVariable Long id) {
        userService.deleteUser(id);
    }
}
```

5.4 Custom Query Methods

You can define custom methods in the repository using keywords:

Listing 5: Custom Query Methods

```
List<User> findByName(String name);
User findByEmail(String email);
```

5.5 Spring Boot Application Setup

application.properties configuration:

Listing 6: application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=your_password
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Main class to run the app:

Listing 7: Main Application

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.
   SpringBootApplication;

@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

5.6 Summary

- Spring Data JPA simplifies data access using interfaces.
- No need to implement repository methods manually.
- Entities are Java classes mapped to database tables.
- Custom queries can be generated using method names.

6 Spring Boot Validation

Validation is essential to ensure that the data coming from users or APIs is correct and safe before it reaches your business logic or database.

Spring Boot makes validation easy using annotations from the `jakarta.validation` package (previously `javax.validation`).

6.1 Add Dependencies

If you're using Maven, make sure the following dependency is in your `pom.xml`:

Listing 8: pom.xml - Validation Dependency

```
<dependency>
    <groupId>jakarta.validation</groupId>
    <artifactId>jakarta.validation-api</artifactId>
    <version>3.0.2</version>
```

```
</dependency>
```

(Spring Boot Starter Web includes Hibernate Validator by default.)

6.2 Annotate Your Entity

Use validation annotations like `@NotNull`, `@Email`, `@Size`, etc., to validate input fields.

Listing 9: User Entity with Validation

```
import jakarta.persistence.*;
import jakarta.validation.constraints.*;

@Entity
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Name is required")
    private String name;

    @Email(message = "Email must be valid")
    @NotBlank(message = "Email is required")
    private String email;

    @Min(value = 18, message = "Must be at least 18 years old")
    private int age;

    // Getters and Setters
}
```

6.3 Update the Controller

To validate incoming requests, annotate the method parameter with `@Valid` and handle errors using `BindingResult`.

Listing 10: User Controller with Validation

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;
import org.springframework.validation.BindingResult;
import java.util.List;

@RestController
@RequestMapping("/users")
public class UserController {

    @Autowired
    private UserService userService;
```

```
    @PostMapping
    public String createUser(@Valid @RequestBody User user,
        BindingResult result) {
        if (result.hasErrors()) {
            return result.getAllErrors().get(0).getDefaultMessage
                (); // Return first error
        }
        userService.createUser(user);
        return "User created successfully!";
    }
}
```

6.4 Common Validation Annotations

- `@NotNull` – Field must not be `null`.
- `@NotBlank` – Field must not be empty or whitespace.
- `@Size(min=, max=)` – Field must have length within the range.
- `@Email` – Validates email format.
- `@Min`, `@Max` – Validate numeric ranges.
- `@Pattern` – Validate format using regular expressions.

6.5 Optional: Global Error Handling

You can handle validation errors globally using `@ControllerAdvice`.

Listing 11: Global Exception Handler

```
import org.springframework.web.bind.
   MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;
import java.util.*;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public Map<String, String> handleValidation(
        MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error ->
            errors.put(error.getField(), error.getDefaultMessage
                ())
        );
        return errors;
    }
}
```

7 Spring Boot Actuator

Spring Boot Actuator adds production-ready features to your application. It helps you monitor and manage your app by exposing various REST endpoints related to health, metrics, environment, logs, and more.

7.1 Adding the Actuator Dependency

Add the following to your `pom.xml` if you're using Maven:

Listing 12: pom.xml - Actuator Dependency

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

7.2 Enable Actuator Endpoints in `application.properties`

By default, only the `/actuator/health` and `/actuator/info` endpoints are enabled.
You can enable more:

Listing 13: application.properties Configuration

```
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
```

7.3 Accessing Actuator Endpoints

Once Actuator is added and configured, you can access endpoints via browser or Postman.

- `/actuator` — Lists all available endpoints.
- `/actuator/health` — Checks app health.
- `/actuator/info` — Displays info you configure.
- `/actuator/metrics` — Shows performance metrics (memory, threads, CPU, etc).
- `/actuator/env` — Displays environment variables and config values.
- `/actuator/loggers` — Shows log configuration.

7.4 Customizing Info Endpoint

You can provide custom info values in `application.properties`:

Listing 14: Custom Info

```
info.app.name=My Spring Boot App
info.app.description=A sample application with actuator
info.app.version=1.0.0
```

When you call `/actuator/info`, it will return:

Listing 15: Response from /actuator/info

```
{
  "app": {
    "name": "My Spring Boot App",
    "description": "A sample application with actuator",
    "version": "1.0.0"
  }
}
```

7.5 Health Endpoint Details

`/actuator/health` can show checks for:

- Database connectivity
- Disk space
- Custom health indicators

Example response:

Listing 16: Response from /actuator/health

```
{
  "status": "UP",
  "components": {
    "db": {
      "status": "UP",
      "details": {
        "database": "MySQL",
        "result": "OK"
      }
    },
    "diskSpace": {
      "status": "UP",
      "details": {
        "total": 499963174912,
        "free": 350000000000,
        "threshold": 10485760
      }
    }
  }
}
```

7.6 Custom Health Indicator

You can define your own health checks:

Listing 17: Custom Health Indicator

```
import org.springframework.boot.actuate.health.Health;
import org.springframework.boot.actuate.health.HealthIndicator;
import org.springframework.stereotype.Component;

@Component
public class CustomHealthIndicator implements HealthIndicator {

    @Override
    public Health health() {
        boolean healthy = checkMyService(); // Your logic here
        if (healthy) {
            return Health.up().withDetail("service", "Available")
               .build();
        } else {
            return Health.down().withDetail("service", "Down").
               build();
        }
    }

    private boolean checkMyService() {
        // Simulate custom health check
        return true;
    }
}
```

7.7 Securing Actuator Endpoints

In production, it's important to protect Actuator endpoints. You can secure them using Spring Security:

Listing 18: Restrict Actuator to Admin Only

```
management.endpoints.web.exposure.include=health,info
management.endpoints.web.exposure.exclude=env,beans

# Security configuration (if Spring Security is added)
spring.security.user.name=admin
spring.security.user.password=secret
```

Chapter 5: Spring Security with JWT Authentication

Spring Security provides a comprehensive and customizable authentication and access-control framework for Spring applications. When combined with JWT (JSON Web Token), it allows for stateless, token-based authentication, which is ideal for RESTful APIs.

5.1 Overview of JWT Authentication

JWT is a compact, URL-safe token format that encodes claims between two parties. It's commonly used for user authentication and authorization.

- **Header**: Specifies the algorithm used to sign the token.
- **Payload**: Contains the claims (e.g., username, roles).
- **Signature**: Used to verify the token's authenticity.

5.2 Steps to Implement JWT Authentication in Spring Boot

1. Add dependencies
2. Configure UserDetailsService
3. Create JWT utility class
4. Implement authentication and authorization filters
5. Set up security configuration
6. Test with login and protected endpoints

5.3 Add Dependencies

In `pom.xml`, add:

Listing 19: JWT and Spring Security Dependencies

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt</artifactId>
    <version>0.9.1</version>
</dependency>
```

5.4 Create User Entity and Repository

Listing 20: User Entity

```
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String username;
    private String password;
    private String role;

    // Getters and Setters
}
```

Listing 21: User Repository

```
public interface UserRepository extends JpaRepository<User, Long>
    {
    Optional<User> findByUsername(String username);
}
```

5.5 Implement UserDetailsService

Listing 22: Custom UserDetailsService

```
@Service
public class CustomUserDetailsService implements
   UserDetailsService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws
        UsernameNotFoundException {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("
               User not found"));

        return new org.springframework.security.core.userdetails.
           User(
            user.getUsername(), user.getPassword(),
            Collections.singletonList(new SimpleGrantedAuthority(
               user.getRole())));
    }
}
```

5.6 JWT Utility Class

Listing 23: JWT Utility

```
@Component
public class JwtUtil {

    private String secret = "mySecretKey";

    public String generateToken(String username) {
        return Jwts.builder()
                .setSubject(username)
                .setIssuedAt(new Date())
                .setExpiration(new Date(System.currentTimeMillis
                   () + 86400000)) // 1 day
                .signWith(SignatureAlgorithm.HS512, secret)
                .compact();
    }

    public String extractUsername(String token) {
        return Jwts.parser().setSigningKey(secret).parseClaimsJws
           (token)
                .getBody().getSubject();
    }

    public boolean validateToken(String token, UserDetails
       userDetails) {
        String username = extractUsername(token);
        return username.equals(userDetails.getUsername()) &&
               !isTokenExpired(token);
    }

    private boolean isTokenExpired(String token) {
        Date expiration = Jwts.parser().setSigningKey(secret)
                .parseClaimsJws(token).getBody().getExpiration();
        return expiration.before(new Date());
    }
}
```

5.7 Authentication Filter

Listing 24: JWT Authentication Filter

```
public class JwtAuthenticationFilter extends
   UsernamePasswordAuthenticationFilter {

    private AuthenticationManager authenticationManager;
    private JwtUtil jwtUtil;

    public JwtAuthenticationFilter(AuthenticationManager
       authenticationManager, JwtUtil jwtUtil) {
```

```
        this.authenticationManager = authenticationManager;
        this.jwtUtil = jwtUtil;
        setFilterProcessesUrl("/login");
    }

    @Override
    public Authentication attemptAuthentication(
       HttpServletRequest request, HttpServletResponse response)
       {
        try {
            LoginRequest creds = new ObjectMapper().readValue(
               request.getInputStream(), LoginRequest.class);
            return authenticationManager.authenticate(
                    new UsernamePasswordAuthenticationToken(creds
                       .getUsername(), creds.getPassword()));
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }

    @Override
    protected void successfulAuthentication(HttpServletRequest
       request, HttpServletResponse response,
                                            FilterChain chain,
                                               Authentication
                                               authResult) throws
                                                IOException {
        String username = ((User) authResult.getPrincipal()).
           getUsername();
        String token = jwtUtil.generateToken(username);
        response.addHeader("Authorization", "Bearer " + token);
    }
}
```

5.8 Authorization Filter

Listing 25: JWT Authorization Filter

```
public class JwtAuthorizationFilter extends
   BasicAuthenticationFilter {

    private JwtUtil jwtUtil;
    private CustomUserDetailsService userDetailsService;

    public JwtAuthorizationFilter(AuthenticationManager
       authenticationManager, JwtUtil jwtUtil,
                                  CustomUserDetailsService
                                     userDetailsService) {
        super(authenticationManager);
        this.jwtUtil = jwtUtil;
        this.userDetailsService = userDetailsService;
```

```
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request,
       HttpServletResponse response,
                                    FilterChain chain) throws
                                       IOException,
                                       ServletException {
        String header = request.getHeader("Authorization");

        if (header == null || !header.startsWith("Bearer ")) {
            chain.doFilter(request, response);
            return;
        }

        String token = header.replace("Bearer ", "");
        String username = jwtUtil.extractUsername(token);

        if (username != null) {
            UserDetails userDetails = userDetailsService.
               loadUserByUsername(username);
            if (jwtUtil.validateToken(token, userDetails)) {
                UsernamePasswordAuthenticationToken auth = new
                   UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.
                           getAuthorities());
                SecurityContextHolder.getContext().
                   setAuthentication(auth);
            }
        }
        chain.doFilter(request, response);
    }
}
```

5.9 Security Configuration

Listing 26: Security Configuration

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter
   {

    @Autowired
    private CustomUserDetailsService userDetailsService;

    @Autowired
    private JwtUtil jwtUtil;

    @Override
```

```
    protected void configure(HttpSecurity http) throws Exception
       {
        AuthenticationManager authenticationManager =
           authenticationManager();
        http.csrf().disable()
            .authorizeRequests()
            .antMatchers("/login").permitAll()
            .anyRequest().authenticated()
            .and()
            .addFilter(new JwtAuthenticationFilter(
               authenticationManager, jwtUtil))
            .addFilter(new JwtAuthorizationFilter(
               authenticationManager, jwtUtil, userDetailsService
               ));
    }

    @Override
    protected void configure(AuthenticationManagerBuilder auth)
       throws Exception {
        auth.userDetailsService(userDetailsService)
            .passwordEncoder(new BCryptPasswordEncoder());
    }

    @Bean
    @Override
    public AuthenticationManager authenticationManagerBean()
       throws Exception {
        return super.authenticationManagerBean();
    }
}
```

5.10 Summary

- JWT enables stateless authentication, perfect for REST APIs.
- Spring Security integrates well with JWT through custom filters.
- Authentication: verifies user credentials and returns a token.
- Authorization: filters requests and checks the token validity.

This setup secures your backend with JWT, ensuring only authenticated users can access protected endpoints.

Chapter 6: Exception Handling & Logging

Building robust applications means handling errors gracefully and keeping track of what's happening in the system. In Spring Boot, we achieve this through:

- Global Exception Handling using `@ControllerAdvice`
- Custom Exception classes
- Logging using `SLF4J`, `Logback`, or other libraries

6.1 Why Exception Handling Matters

Exception handling improves the user experience and application stability. Instead of returning confusing or sensitive error messages, we can show clear messages and log details for developers.

6.2 Creating a Custom Exception

Let's define a custom exception class:

Listing 27: Custom NotFoundException

```
public class NotFoundException extends RuntimeException {
    public NotFoundException(String message) {
        super(message);
    }
}
```

6.3 Throwing the Exception

You can throw this exception in the service layer when data is not found:

Listing 28: Throwing Custom Exception

```
public User getUserById(Long id) {
    return userRepository.findById(id)
        .orElseThrow(() -> new NotFoundException("User not found
           with id: " + id));
}
```

6.4 Handling Exceptions Globally

Instead of handling exceptions inside every controller, Spring allows global exception handling using `@ControllerAdvice` and `@ExceptionHandler`.

Listing 29: Global Exception Handler

```
import org.springframework.http.*;
import org.springframework.web.bind.annotation.*;

@ControllerAdvice
```

```
public class GlobalExceptionHandler {

    @ExceptionHandler(NotFoundException.class)
    public ResponseEntity<String> handleNotFoundException(
       NotFoundException ex) {
        return new ResponseEntity<>(ex.getMessage(), HttpStatus.
           NOT_FOUND);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handleGeneralException(
       Exception ex) {
        return new ResponseEntity<>("Something went wrong",
           HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

6.4.1 Returning JSON Responses

You can return structured JSON error responses instead of plain text:

Listing 30: Structured Error Response

```
public class ErrorResponse {
    private String message;
    private LocalDateTime timestamp;

    public ErrorResponse(String message) {
        this.message = message;
        this.timestamp = LocalDateTime.now();
    }

    // Getters and Setters
}
```

Update the global handler:

Listing 31: Updated Global Handler with JSON

```
@ExceptionHandler(NotFoundException.class)
public ResponseEntity<ErrorResponse> handleNotFoundException(
   NotFoundException ex) {
    ErrorResponse error = new ErrorResponse(ex.getMessage());
    return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
}
```

6.5 Logging in Spring Boot

Logging is essential for debugging and monitoring. Spring Boot uses SLF4J with Logback as the default logging framework.

6.5.1 Add Logger to Your Class

Use SLF4J:

Listing 32: Using Logger

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Service
public class UserService {

    private static final Logger logger = LoggerFactory.getLogger(
       UserService.class);

    public User getUserById(Long id) {
        logger.info("Fetching user with id {}", id);
        return userRepository.findById(id)
            .orElseThrow(() -> new NotFoundException("User not
               found"));
    }
}
```

6.5.2 Logging Levels

SLF4J supports different logging levels:

- `TRACE` — Most detailed (used for debugging).
- `DEBUG` — Debug messages for development.
- `INFO` — General application flow information.
- `WARN` — Something unexpected, but not an error.
- `ERROR` — An error occurred.

6.6 Changing Log Level in application.properties

Listing 33: application.properties Logging Settings

```
# Show only INFO and above
logging.level.root=INFO

# Set DEBUG level for your service
logging.level.com.example.service=DEBUG
```

6.7 Example Error Response

Here's how a structured error might look in the frontend:

Listing 34: JSON Error Response

```
{
  "message": "User not found with id: 5",
  "timestamp": "2025-08-08T15:23:45.123"
}
```

6.8 Summary

- Use `@ControllerAdvice` for global exception handling.
- Define custom exceptions to represent specific error cases.
- Use structured error responses to communicate clearly with the client.
- Use SLF4J for logging and monitor app behavior using logs.
- Control logging verbosity via `application.properties`.